

Rapport strategique

Publicite, revenus et couts d'infrastructure

FinHub - analyse au 15 mars 2026

1. Objet du rapport

Ce rapport a un objectif simple: expliquer a une personne exterieure au projet comment FinHub peut monetiser proprement son audience, quels revenus publicitaires sont plausibles selon plusieurs scenarios, et a quel moment l'architecture Firestore actuelle commence a couter plus cher qu'elle ne le devrait.

Le fil rouge est le suivant:

- FinHub n'est pas une app de contenu passive. C'est une app d'apprentissage gamifie de la finance, avec un melange de pedagogie, de boucles quotidiennes, d'economie virtuelle, de portefeuille simule et de social.
- Cette nature hybride change completement la strategie pub. Les emplacements rentables ne sont pas les memes que dans un jeu mobile pur, et les endroits ou une publicite detruit la confiance ne sont pas les memes que dans une app media.
- Cote couts, Firestore reste globalement peu cher tant que les requetes restent locales et bien bornees. En revanche, certaines ecoutes temps reel sur des collections larges ou non limitees creent un risque de derive economique a mesure que l'app grandit.
- Enfin, deplacer les workers GitHub Actions vers Cloud Functions / Cloud Scheduler peut simplifier l'exploitation et reduire un petit cout de compute, mais cela ne resout pas a lui seul les vrais postes de depense. Le point critique reste la forme des requetes Firestore.

2. Lecture produit: ou l'app cree de la valeur, et donc ou la pub est acceptable

FinHub repose aujourd'hui sur cinq surfaces principales:

- 'Aujourd'hui': hub quotidien, relance, actus, notifications, raccourcis.
- 'Dashboard': portefeuille et suivi de positions.
- 'Apprendre': cours, quiz, progression.
- 'Game': recompenses quotidiennes, simulation, quetes, economie virtuelle.
- 'Social': communaute, forum, amis, duels, classements.

En termes de perception utilisateur, ces surfaces n'ont pas la meme tolerance a la publicite:

- Les zones de confiance forte sont 'Dashboard', 'Apprendre', les vues portefeuille et les ecrans d'information financiere. Ici, la pub intrusive detruit la credibilite du produit.
- Les zones de boucle et de monnaie virtuelle sont 'Game', 'Daily reward', 'Daily News', 'rerolls', 'bonus', 'quetes', 'duels'. Ici, une pub recompensee peut etre percue comme un echange explicite et acceptable.
- Les zones de type feed sont 'Aujourd'hui', 'Social', 'Forum', 'Notifications'. Ici, des formats natifs legers peuvent fonctionner s'ils restent visuellement propres et clairement marques comme sponsorises.

Conclusion produit:

- FinHub doit être monétisée en priorité par des formats 'rewarded' optionnels.
- Les 'native ads' ne sont acceptables que dans les surfaces feed / hub.
- Les 'interstitials' doivent rester rares et placés uniquement sur des transitions naturelles de fin de session.
- Les bannières fixes ne sont pas recommandées: elles dégradent l'expérience plus qu'elles ne rapportent dans un produit de ce type.

3. Ce que le code montre aujourd'hui

L'audit du code confirme cette lecture produit mais montre aussi plusieurs signaux de cout:

- Le hub 'Aujourd'hui' écoute en temps réel plusieurs documents utiles, mais aussi deux collections entières: 'metalsPrices' et 'broadcasts'.
- L'onglet social charge un classement global à partir d'un listener sur toute la collection 'users', ce qui est acceptable à petite échelle mais dangereux à grande échelle.
- Plusieurs écrans cumulent des listeners imbriqués sur les mêmes documents utilisateur, ce qui augmente le nombre de reads initiaux.
- Le tracking d'activité écrit deux documents par événement tracké.
- Les jobs backend actuels tournent via GitHub Actions selon une cadence très fréquente, surtout pour les duels.

Reperes techniques internes utilisés pour cette analyse:

Today page

- lib/pages/today_page.dart:202-207 -> listener users/{uid}
- lib/pages/today_page.dart:1730-1758 -> listeners progress, quests, dailyBenefits, metalsPrices/{today}, metalsPrices (collection entière), broadcasts (collection entière)

Social / leaderboard

- lib/features/social/social_spotlight_card.dart:87-88 -> collection('users').snapshots()
- lib/features/social/social_spotlight_card.dart:460-463 -> ce flux alimente le leaderboard

Notifications

- lib/features/notifications/notification_center_page.dart:309-332 -> 4 listeners principaux
- lib/features/notifications/notification_center_page.dart:766-840 -> listeners additionnels par item actionable

Tracking d'activité

- lib/services/activity_tracking_service.dart:44-68 -> 1 write sur users/{uid}/activity_daily
- + 1 write sur users/{uid} pour last_active_at

Workers planifiés

- .github/workflows/daily_metals.yml:4-8
- .github/workflows/weekly_leaderboard.yml:4-5
- .github/workflows/settle.yml:4-5

Leaderboard batch

- scripts/build_weekly_leaderboard.js:295-300 -> lecture de tous les users puis chargement d'activité par user

4. Scénarios publicitaires

Principes de conception

Avant de parler chiffres, voici la règle de décision utilisée:

- On ne place pas de pub dans une étape où l'utilisateur est en train d'apprendre, de réfléchir ou de consulter son portefeuille.
- On place la pub là où l'utilisateur comprend immédiatement l'échange: "je regarde une pub, j'obtiens un bonus".

- On garde une fréquence ciblée et lisible pour éviter l'effet casino.
- On privilégie d'abord la rétention et la confiance; la monétisation vient ensuite.

Scenario A - Conservateur

Positionnement:

- Bonus après le cadeau quotidien gratuit.
- Une session 'Daily News' sponsorisée après l'entrée gratuite.
- Un bonus ponctuel sur une mécanique de reroll / double récompense dans 'Game'.

Formats:

- 'Rewarded ads' uniquement.

Impact UX:

- Génération faible.
- Très bon respect de la promesse éducative.
- Monétisation plus lente mais saine.

Projection de revenu brut:

Hypothèse de monétisation

- 0,35 impression rewarded / DAU / jour
- eCPM rewarded iOS Europe de l'Ouest: 8 \$ à 12 \$

Projection brute

- 1k DAU: 84 \$ à 126 \$ / mois
- 10k DAU: 840 \$ à 1 260 \$ / mois
- 50k DAU: 4 200 \$ à 6 300 \$ / mois
- 100k DAU: 8 400 \$ à 12 600 \$ / mois

Scenario B - Equilibre et recommande

Positionnement:

- Tout le scénario A.
- Une 'native ad' dans le hub 'Aujourd'hui', sous le premier bloc utile.
- Une 'native ad' dans l'espace 'Social / Forum'.
- Un 'interstitial' très rare, uniquement après la fin complète d'une session fermée: fin de run 'Daily News', fin de simulation, fin de séquence de duel consommée.

Formats:

- 'Rewarded' en priorité.
- 'Native' léger sur les surfaces feed.
- 'Interstitial' ciblé très strictement.

Impact UX:

- Génération faible à modérée.
- Bon compromis revenu / image produit.
- C'est le scénario qui semble le plus cohérent avec FinHub aujourd'hui.

Projection de revenu brut:

Hypothèse de monétisation

- 0,45 rewarded / DAU / jour
- 1,20 native / DAU / jour
- 0,10 interstitial / DAU / jour

- eCPM rewarded: 8 \$ a 12 \$
 - eCPM native: 1 \$ a 2 \$
 - eCPM interstitial: 4 \$ a 7 \$
- Projection brute
- 1k DAU: 156 \$ a 255 \$ / mois
 - 10k DAU: 1 560 \$ a 2 550 \$ / mois
 - 50k DAU: 7 800 \$ a 12 750 \$ / mois
 - 100k DAU: 15 600 \$ a 25 500 \$ / mois

Scenario C - Croissance plus agressive

Positionnement:

- Tout le scenario B.
- Un 'app open ad' tres doux au retour au foreground, pas au cold start.
- Plus de transitions monetisees autour des runs de jeu et des fins de session.

Formats:

- 'Rewarded'
- 'Native'
- 'Interstitial'
- 'App Open'

Impact UX:

- Gene moderee.
- Augmentation sensible du risque de fatigue publicitaire.
- A reserver a un produit deja tres stable en retention.

Projection de revenu brut:

- Hypothese de monetisation
- 0,65 rewarded / DAU / jour
 - 1,60 native / DAU / jour
 - 0,22 interstitial / DAU / jour
 - 0,20 app open / DAU / jour
 - eCPM rewarded: 8 \$ a 12 \$
 - eCPM native: 1 \$ a 2,2 \$
 - eCPM interstitial: 4 \$ a 7 \$
 - eCPM app open: 3 \$ a 6 \$
- Projection brute
- 1k DAU: 248 \$ a 422 \$ / mois
 - 10k DAU: 2 484 \$ a 4 218 \$ / mois
 - 50k DAU: 12 420 \$ a 21 090 \$ / mois
 - 100k DAU: 24 840 \$ a 42 180 \$ / mois

Recommandation publicitaire

La meilleure trajectoire n'est pas le scenario le plus agressif.

La recommandation est:

- court terme: Scenario A si l'objectif principal est de preserver la qualite produit et de valider la mecanique rewarded;
- trajectoire recommandee: Scenario B, parce qu'il monetise les surfaces "naturellement sponsorisables" sans fragiliser les surfaces de confiance;
- eviter le scenario C tant que la retention, les caps de frequence et la mesure de churn post-pub ne sont pas solides.

5. Modele de cout Firestore

Ce que facture Firestore

Sur la base des tarifs standard Firestore cites dans la documentation Firebase, le modele utilise dans ce rapport est:

```
Prix unitaires utilises
- Read: 0,06 $ / 100 000 reads
- Write: 0,18 $ / 100 000 writes
- Delete: 0,02 $ / 100 000 deletes
Soit en cout unitaire
- 1 read = 0,0000006 $
- 1 write = 0,0000018 $
- 1 delete = 0,0000002 $
```

Important:

- Les listeners temps reel sont factures a l'initialisation puis a chaque document ajoute, modifie ou retire du resultat.
- Si un listener se reconnecte apres une coupure suffisamment longue, Firestore le refacture comme une nouvelle requete.
- La documentation officielle recommande explicitement de garder les datasets ecoutes en temps reel petits.

Hypothese de modelisation

Ce rapport ne s'appuie pas sur des analytics de production. Il s'appuie sur le code actuel et sur des hypotheses d'usage plausibles.

J'utilise donc trois profils de "journee utilisateur":

- 'leger': ouvre l'app, consulte le hub, recupere un bonus, fait peu d'actions;
- 'standard': utilise le hub, le jeu, une partie apprentissage, quelques recompenses et ecrit un peu de progression;
- 'intense': utilise plusieurs surfaces, plus de quetes, plus de tracking, plus de navigation, plus de social.

Projection Firestore du cout mensuel, hors stockage et bande passante:

```
Profil leger
- 250 reads / jour / utilisateur
- 12 writes / jour / utilisateur
- 0,2 delete / jour / utilisateur
```

Projection mensuelle

```
- 1k DAU: 5,15 $ / mois
- 10k DAU: 51,49 $ / mois
- 50k DAU: 257,46 $ / mois
- 100k DAU: 514,92 $ / mois
```

Profil standard

```
- 650 reads / jour / utilisateur
- 20 writes / jour / utilisateur
- 0,4 delete / jour / utilisateur
```

Projection mensuelle

```
- 1k DAU: 12,78 $ / mois
- 10k DAU: 127,82 $ / mois
- 50k DAU: 639,12 $ / mois
- 100k DAU: 1 278,24 $ / mois
```

Profil intense

```
- 1 500 reads / jour / utilisateur
- 35 writes / jour / utilisateur
```

- 1 delete / jour / utilisateur
- Projection mensuelle
- 1k DAU: 28,90 \$ / mois
 - 10k DAU: 288,96 \$ / mois
 - 50k DAU: 1 444,80 \$ / mois
 - 100k DAU: 2 889,60 \$ / mois

Lecture importante:

- A petite et moyenne echelle, Firestore n'est pas le probleme principal de FinHub.
- Le cout variable pur par utilisateur est bas.
- Le vrai danger n'est pas "Firestore est cher"; le vrai danger est "certaines requetes ont une forme qui devient chere quand la taille des collections augmente".

6. Les points de derive a grande echelle

6.1 Hub 'Aujourd'hui': ecoute de collections entieres

Aujourd'hui, le hub ecoute:

- le document du jour sur 'metalsPrices/{today}';
- mais aussi toute la collection 'metalsPrices';
- et toute la collection 'broadcasts'.

Ce design fonctionne au debut, mais sa facture augmente avec la taille de chaque collection.

Hypothese utilisee pour illustrer le risque:

- 'metalsPrices': 365 documents d'historique;
- 'broadcasts': 80 documents, car le script de notification les prune a 80;
- 2 sessions utilisateur par jour qui declenchent un snapshot initial.

Impact mensuel des seuls listeners "historique + broadcasts":

- Cout mensuel additionnel estime
- 10k DAU: 160,20 \$ / mois
 - 50k DAU: 801,00 \$ / mois
 - 100k DAU: 1 602,00 \$ / mois

Ce n'est pas dramatique a 10k DAU, mais cela devient un cout inutile parce que la valeur produit n'exige pas de relire l'integralite de l'historique pour afficher le hub du jour.

6.2 Social leaderboard: listener global sur 'users'

C'est le point le plus risque du code actuel.

Le leaderboard du social repose aujourd'hui sur un listener temps reel sur toute la collection 'users'. Tant que la base reste petite, cela passe. Des que le produit grossit, on se retrouve avec un cout proportionnel a:

'nombre de visites social par jour x nombre total d'utilisateurs stockes'

Projection illustrative:

- 10k utilisateurs totaux, 10k DAU, 20% ouvrent le social chaque jour;
- 50k utilisateurs totaux, 50k DAU, 15% ouvrent le social chaque jour;
- 100k utilisateurs totaux, 100k DAU, 10% ouvrent le social chaque jour.

Cout mensuel estime du seul listener global users

- 10k users / 10k DAU / 20% social: 360 \$ / mois
- 50k users / 50k DAU / 15% social: 6 750 \$ / mois
- 100k users / 100k DAU / 10% social: 18 000 \$ / mois

Ce point change complètement l'analyse économique du produit:

- sans correction, un très bon usage social peut détruire une grande partie de la marge publicitaire;
- avec correction, Firestore redevient un coût très raisonnable.

6.3 Tracking d'activité: petit coût unitaire, gros volume cumulé

Chaque événement tracké écrit:

- un document journalier 'users/{uid}/activity_daily/{date}';
- puis met à jour 'users/{uid}.last_active_at'.

Un seul événement coûte peu. Mais sur une app gamifiée, les événements s'empilent vite. Le tracking n'est pas le plus gros risque actuel, mais il devient un accélérateur de ventes à mesure que les mini-boucles se multiplient.

7. Projection après optimisation des requêtes

Si l'on garde le produit tel quel mais qu'on fait seulement une "hygiène de requêtes", le coût chute vite:

- remplacer les listeners globaux par des requêtes limitées / paginées;
- pré-calculer les classements affichés dans le social;
- écouter 'broadcasts' avec 'orderBy(createdAt desc).limit(20)';
- écouter 'metalsPrices' avec une fenêtre bornée ou un document de synthèse;
- dédupliquer certains listeners utilisateur redondants.

Modèle optimisé retenu:

Profil standard optimisé

- 150 reads / jour / utilisateur
- 18 writes / jour / utilisateur
- 0,4 delete / jour / utilisateur

Projection mensuelle

- 1k DAU: 3,67 \$ / mois
- 10k DAU: 36,74 \$ / mois
- 50k DAU: 183,72 \$ / mois
- 100k DAU: 367,44 \$ / mois

Profil intense optimisé

- 300 reads / jour / utilisateur
- 30 writes / jour / utilisateur
- 1 delete / jour / utilisateur

Projection mensuelle

- 1k DAU: 7,03 \$ / mois
- 10k DAU: 70,26 \$ / mois
- 50k DAU: 351,30 \$ / mois
- 100k DAU: 702,60 \$ / mois

Lecture business:

- A 10k DAU, la différence entre état actuel standard et état optimisé reste faible en valeur absolue.
- A 50k ou 100k DAU, l'optimisation devient structurelle parce qu'elle protège la marge.
- L'optimisation la plus urgente n'est pas une micro-optimisation; c'est la suppression des listeners non bornés.

8. GitHub Actions aujourd'hui, Cloud Functions demain

Ce qui tourne actuellement

Aujourd'hui, trois familles de jobs planifiés existent:

```
Cadence observee
- Daily metals:          24 runs / jour
- Weekly leaderboard:    1 run / jour
- Duel worker settle:    144 runs / jour
Total estime: 169 runs / jour
Total estime: 5 070 runs / mois
```

Cout GitHub Actions actuel

Le cout actuel depend d'un point externe au repo:

- si le depot est public: les runners standards GitHub-hosted sont gratuits;
- si le depot est prive: le cout depend du plan GitHub et des minutes incluses.

Reperes officiels actuels:

- GitHub Free: 2 000 minutes / mois incluses;
- GitHub Pro / Team: 3 000 minutes / mois incluses;
- Linux 2-core standard runner: 0,006 \$ / minute depuis le 1er janvier 2026.

Avec 5 070 runs / mois, le cout depend donc surtout de la duree moyenne de chaque job:

```
Si le repo est prive et si les jobs utilisent un runner Linux 2-core
Duree moyenne 0,75 min / run
- 3k minutes incluses: ~ 4,82 $ / mois au-dela du quota
Duree moyenne 1,00 min / run
- 3k minutes incluses: ~ 12,42 $ / mois au-dela du quota
Duree moyenne 1,50 min / run
- 3k minutes incluses: ~ 27,63 $ / mois au-dela du quota
```

En clair:

- aujourd'hui, GitHub Actions ne represente probablement qu'un petit cout, ou meme zero si le repo est public;
- ce n'est pas le poste le plus important.

Ce que change le passage a Cloud Functions / Cloud Scheduler

Si par "passer les actions git en CF" on entend "remplacer ces workers GitHub Actions par des fonctions planifiees Firebase / Google Cloud", alors:

- 'Cloud Scheduler': les 3 premiers jobs sont inclus sans cout, puis 0,10 \$ par job et par mois;
- 'Cloud Functions': 2M invocations / mois incluses au Blaze plan;
- 'FCM': toujours sans cout d'usage.

Avec les cadences actuelles:

- 5 070 invocations / mois est tres loin du seuil gratuit de 2M invocations;
- 3 jobs planifies tiennent dans l'allocation gratuite Cloud Scheduler;
- le cout direct de compute des jobs actuels serait donc probablement proche de zero a ce stade, ou tres faible.

Le point cle

Deplacer ces workers vers Cloud Functions ameliore:

- la fiabilite d'exploitation;
- la centralisation des secrets;
- la coherence de la stack Firebase;
- la possibilite de brancher des triggers plus propres a terme.

Mais cela ne change pas la logique economique fondamentale:

- si un job lit beaucoup de documents Firestore, il les lira toujours en Cloud Functions;
- si un leaderboard charge tous les users, le cout Firestore restera lie au nombre de users, quelle que soit la plateforme d'execution;
- migrer GitHub Actions vers Cloud Functions ne remplace donc pas l'optimisation des requetes Firestore.

Nuance de cout 2nd gen

Il faut cependant noter une subtilite officielle importante:

- en Cloud Functions 2nd gen, les petites fonctions low-memory ont par default 1 CPU, ce qui peut rendre certaines fonctions plus cheres qu'attendu si on les porte sans réglage fin.

Dans votre cas, cette nuance ne renverse pas la conclusion:

- a l'echelle actuelle, les workers resteront vraisemblablement dans les quotas gratuits ou tres proches;
- a plus grande echelle, la vraie facture restera largement cote Firestore et non cote invocations de fonctions.

9. Lecture economique globale

Le point le plus interessant du dossier est le suivant:

- dans une version disciplinee de FinHub, la pub peut largement couvrir les couts Firestore;
- dans une version non optimisee avec listeners globaux, la marge peut etre rongee inutilement a mesure que le social grossit.

Lecture synthetique:

A 10k DAU

- Revenu pub scenario B: 1 560 \$ a 2 550 \$ / mois
- Firestore etat standard: 128 \$ / mois
- Firestore optimise: 37 \$ / mois

A 50k DAU

- Revenu pub scenario B: 7 800 \$ a 12 750 \$ / mois
- Firestore etat standard: 639 \$ / mois
- Firestore optimise: 184 \$ / mois
- Risque si leaderboard global users conserve: ~ 6 750 \$ / mois a lui seul

A 100k DAU

- Revenu pub scenario B: 15 600 \$ a 25 500 \$ / mois
- Firestore etat standard: 1 278 \$ / mois
- Firestore optimise: 367 \$ / mois
- Risque si leaderboard global users conserve: ~ 18 000 \$ / mois

La conclusion n'est donc pas "la pub financera forcément tout" ni "Firestore est trop cher". La bonne conclusion est:

- le modele peut etre tres rentable;
- mais cette rentabilite depend fortement de deux ou trois decisions de requetes qui paraissent anodines tant que la base est petite.

10. Recommandations concretes

Priorite 1 - Monetisation

Lancer d'abord le scenario B, pas le C:

- bonus rewarded apres le cadeau quotidien;
- une session sponsorisee 'Daily News';
- une native dans 'Aujourd'hui';
- une native dans 'Social / Forum';
- un interstitial tres rare a une vraie fin de session.

Priorite 2 - Requetes Firestore a corriger avant la vraie montee en charge

1. Remplacer le listener global 'users' du social par:
 - un leaderboard materialise;
 - ou une collection dediee 'leaderboards/current';
 - ou au minimum une requete 'limit(10)' / pagination.
2. Remplacer les listeners entiers sur 'metalsPrices' et 'broadcasts' par:
 - un document de synthese du jour;
 - ou une requete ordonnee et bornee.
3. Reduire les listeners imbriques redondants sur 'users/{uid}' dans 'Game', 'Shop' et certaines cartes de hub.

Priorite 3 - Backend ops

Migrer les jobs GitHub Actions vers Cloud Functions / Cloud Scheduler pour:

- simplifier l'exploitation;
- eviter la dependance a GitHub pour des taches produit;
- preparer des traitements backend plus proches du reste de l'architecture.

Mais ne pas vendre cette migration comme une economie massive:

- la vraie economie viendra surtout de l'hygiene Firestore.

11. Conclusion

FinHub a un profil tres favorable a une monetisation publicitaire mesuree:

- la boucle quotidienne et les monnaies virtuelles se pretent bien au rewarded;
- les surfaces feed se pretent a quelques native ads;
- les surfaces de confiance doivent rester tres peu polluees.

Le meilleur compromis, a ce stade, est un modele 'rewarded-first' complete par un peu de 'native', avec tres peu d'interstitiels.

Sur les couts, le constat est rassurant mais exigeant:

- Firestore pur reste peu cher par utilisateur si les requetes sont bien formees;
- l'architecture actuelle contient deja un ou deux patterns qui ne passeront pas bien a grande echelle;
- corriger ces patterns avant d'accelerer l'acquisition est une decision de marge, pas seulement une decision d'ingenierie.

Autrement dit:

- Oui, la pub peut financer proprement l'app.
- Oui, passer les workers GitHub vers Cloud Functions a du sens.
- Non, il ne faut pas confondre migration d'execution et vraie optimisation de cout.
- Le vrai levier economique prioritaire est la suppression des listeners non bornes et des lectures globales inutiles.

12. Sources

Sources officielles:

- Firebase pricing: <https://firebase.google.com/pricing>
- Cloud Firestore billing: <https://firebase.google.com/docs/firestore/pricing>
- Firestore realtime queries at scale:
https://firebase.google.com/docs/firestore/real-time_queries_at_scale
- Scheduled functions / Cloud Scheduler cost note:
<https://firebase.google.com/docs/functions/schedule-functions>
- Cloud Functions for Firebase: <https://firebase.google.com/docs/functions/>
- Cloud Functions manage functions:
<https://firebase.google.com/docs/functions/manage-functions>
- GitHub Actions billing:
<https://docs.github.com/en/billing/managing-billing-for-github-actions/about-billing-for-github-actions>
- GitHub Actions runner pricing:
<https://docs.github.com/billing/reference/actions-runner-pricing>

Benchmarks publicitaires utilises comme base directionnelle:

- Appodeal Benchmarks / eCPM report:
<https://appodeal.com/blog/mobile-ecpm-report-app-ad-monetization-worldwide-performance/>
- Appodeal Q4 2024 report teaser:
https://appodeal.com/wp-content/uploads/2025/03/Quarterly_Mobile_eCPM_Report_-_Q4_2024.pdf

Note methodologique:

- Les placements, taux d'impression et projections de revenu sont des modelisations internes construites a partir du produit, du code et des benchmarks publics; ce ne sont pas des revenus garantis.
- Les projections de cout Firestore sont des ordres de grandeur batis a partir des tarifs officiels et de scenarios d'usage plausibles, pas des exports de facturation de production.